

Development of an Interactive MATLAB Application
for
European Option Pricing and Monte Carlo Simulation

Diego Alberto Ramírez Madriz

December 12, 2025

Contents

1	Introduction	3
1.1	General View On Main Instrument	3
1.2	What is the goal of this project?	3
1.3	Personal Interest:	3
2	Theoretical Background	4
2.1	European Options	4
2.2	Geometric Brownian Motion (GBM)	4
2.3	Black–Scholes Model	4
2.4	Monte Carlo Simulation for Pricing	5
3	Methodology	5
3.1	Discretization of the GBM Process	5
3.2	Monte Carlo Pricing Procedure	5
3.3	Estimation of Statistical Error	6
3.4	Role of Numerical Parameters	6
4	Software Design and Implementation	6
4.1	Overall Architecture	6
4.2	Core Computational Functions	6
4.3	Graphical Application Structure	7
4.4	User Interaction Flow	7
5	Results and Demonstration	8
5.1	Pricing Validation	8
5.2	GBM Path Simulation	8
5.3	Terminal Price Distribution	9
5.4	Graphical Application Demonstration	9
6	Programming Challenges and Solutions	9
6.1	Managing Monte Carlo Performance	10
6.2	Reproducibility and Randomness	10
6.3	Input Validation and Edge Cases	10
6.4	Implementation of Mathematical Models	10
7	Conclusion	11

1 Introduction

1.1 General View On Main Instrument

Financial derivatives are a centerpiece in modern markets, this type of tool allows investors to navigate the stock market with different types of risk and uncertainty, for the scope of this project I will be centering on European Options, a certain type of derivative contract that is simpler to analyze than the American counterpart (in case of doubt, the names do not reflect anything really related to a region) and is quite used as a staple in academic and real trading environments as they let us see how different market conditions influence their value.

1.2 What is the goal of this project?

The goal of this project is to create an interactive application that combines option pricing based on Monte Carlo and Black-Scholes model methods with visualization of price paths and distributions based on Geometric Brownian Motion using Monte Carlo simulations. The idea is to create a tool where the user can change parameters and immediately see how the results or the plots react, this in line with the main part of the project focused on developing a clean and versatile GUI, and accurate mathematical and engineering implementations through good MATLAB practices since this Final Work is meant to demonstrate MATLAB programming skills, not the financial theory itself.

1.3 Personal Interest:

I chose this topic because I have a growing personal interest in quantitative finance, a field in which I would like to specialize in the future, and this project lets me connect the mathematical side with something practical and visual. Following on this, MATLAB seems to provide a good environment for this kind of work, especially because of its focus on fast numerical experiments and easy creation of graphical interfaces through App Designer. To keep the project manageable and within the scope of what is to be reviewed in the subject, I will focus only on European options and assume that the underlying asset, will explain more on what this means in theoretical background, follows a Geometric Brownian Motion model.

The app will include two main modules:

1. an Option Pricer (Black-Scholes and Monte Carlo), and
2. a Monte Carlo Simulator for visualizing asset paths and distributions.

A simple extension for basic portfolio risk evaluation may be added later depending on time and implementation complexity.

2 Theoretical Background

This section introduces the essential concepts and models used in the project. Option prices are computed under specific mathematical assumptions, and understanding these foundations helps clarify the implementation choices made later.

2.1 European Options

An option is a derivative contract that gives its holder the right, but not the obligation, to buy or sell an asset at a predetermined price K (the strike price) on a specific future date T (the maturity). European options can only be exercised on the maturity date, unlike American options which can be exercised at any time before or at T .

There are two basic types of European options:

- **Call option:** gives the holder the right to buy the asset at price K .
- **Put option:** gives the holder the right to sell the asset at price K .

Their payoffs at expiration are given by:

$$\text{Call payoff} = \max(S_T - K, 0), \quad \text{Put payoff} = \max(K - S_T, 0),$$

where S_T is the asset price at maturity.

2.2 Geometric Brownian Motion (GBM)

To model the evolution of the asset price over time, the project assumes a Geometric Brownian Motion, which follows the stochastic differential equation:

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

where μ is the drift, σ is the volatility, and W_t is a standard Brownian motion.

The GBM model has a closed-form solution, which allows simulation of price paths:

$$S_{t+\Delta t} = S_t \exp \left[\left(\mu - \frac{1}{2}\sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \right],$$

with $Z \sim N(0, 1)$.

Under risk-neutral valuation, the drift μ is replaced by the risk-free rate r .

In practice, the continuous-time GBM is discretized using fixed time steps, therefore enabling numerical simulation of asset price paths in the Monte Carlo framework.

2.3 Black–Scholes Model

The Black–Scholes model provides analytical formulas for European call and put prices under the assumptions of no arbitrage, constant volatility, constant interest rate, and GBM dynamics. The formulas are:

$$C = S_0 N(d_1) - K e^{-rT} N(d_2), \quad P = K e^{-rT} N(-d_2) - S_0 N(-d_1),$$

where

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}.$$

For this project, the Black–Scholes price obtained through the formulas is used as a benchmark to assess the accuracy and convergence of the Monte Carlo estimator.

2.4 Monte Carlo Simulation for Pricing

Monte Carlo pricing estimates the expected payoff by simulating a large number of GBM paths and averaging the discounted payoffs. For a call option, the estimator is:

$$\hat{C} = e^{-rT} \cdot \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0),$$

where N is the number of simulated paths.

Monte Carlo methods are flexible and can be extended to more complex payoffs or risk calculations, which is why they form one of the main components of the application.

The statistical uncertainty of the Monte Carlo price is measured through the standard error, defined as the standard deviation of the discounted payoffs that is then divided by the square root of the number of simulated paths. It provides a direct indication of how much the estimate fluctuates due to finite sampling.

3 Methodology

This section describes the numerical procedures used to price European options and to simulate asset price dynamics in the project. The methodology bridges the theoretical models introduced previously with the corresponding practical MATLAB implementation carried out.

3.1 Discretization of the GBM Process

Asset price dynamics are modeled using Geometric Brownian Motion under the a risk neutral measure. The model is initially continuous in time therefore it must be discretized in order to be able to perform numerical simulations.

The time interval $[0, T]$ is divided into n_{steps} equally spaced intervals of length $\Delta t = T/n_{\text{steps}}$. Starting from the initial price S_0 , the asset price is updated iteratively according to

$$S_{t+\Delta t} = S_t \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z_t \right],$$

where $Z_t \sim N(0, 1)$ are independent standard normal random variables. Repeating this procedure generates a full simulated price path.

3.2 Monte Carlo Pricing Procedure

Monte Carlo pricing is carried out by simulating N independent asset price paths under the GBM model described above. Since the option is European, only the terminal value S_T of each path is needed to evaluate the payoff.

For a call option, the discounted payoff associated with the i -th simulated path is given by

$$e^{-rT} \max(S_T^{(i)} - K, 0),$$

with a corresponding expression for a put option. The Monte Carlo estimate of the option price is then obtained by averaging these discounted payoffs over all simulated paths.

3.3 Estimation of Statistical Error

As described in the theoretical background and in addition to the price estimate, the statistical uncertainty of the Monte Carlo method is quantified through the standard error. This is computed as the sample standard deviation of the discounted payoffs divided by the square root of the number of simulated paths. The standard error provides an indication of the variability of the estimator due to finite sampling and decreases as the number of simulations increases.

3.4 Role of Numerical Parameters

Two numerical parameters influence the accuracy and computational cost of the method: the number of time steps used in the GBM discretization and the number of Monte Carlo paths. Increasing the number of steps improves the approximation of the continuous time process, while increasing the number of paths reduces the statistical error of the estimator. In practice, these parameters must be balanced in order to obtain reliable results within reasonable computation time, the default values are supposed to follow this idea.

4 Software Design and Implementation

4.1 Overall Architecture

The software is designed with a clear separation between numerical computation and user interaction. The core pricing and simulation routines are implemented as standalone MATLAB functions, while the graphical interface is responsible for only handling user inputs, triggering computations, and displaying results.

This modular structure improves clarity and maintainability as the computational logic can be tested independently of the graphical application. It also allows the same numerical functions to be reused in different parts of the application, such as the pricing and simulation modules.

The GUI (done using the MATLAB App Designer) acts as a thin orchestration layer. The user may modify the parameters, which are then validated with an internal function, then passed to the implemented mathematical functions. Lastly, the visualization is done through numerical outputs and plots. This design choice ensures that the mathematical models remain decoupled from the presentation layer while providing an interactive and intuitive user experience.

4.2 Core Computational Functions

The application is built around three functions that implement the core numerical models used throughout the project. These functions are written as independent MATLAB function files.

The function `simulate_gbm_paths` is responsible for generating sample paths of the underlying asset price following a Geometric Brownian Motion model. Given the initial price and the corresponding model parameters, it produces a set of simulated trajectories that are later used both for option pricing and for visualization purposes within the application.

Monte Carlo pricing is done by the function `mc_euro_price`, it uses the GBM simulator to generate asset price paths and then focuses on the terminal values of the paths, this because of the option being priced is European. Using this terminal prices the discounted option payoffs is computed, then averaged in order to obtain the Monte Carlo price. The function also computes the standard error obtained by the estimator, this helps us obtain an idea on the variation of the estimator.

The analytical reference price is obtained using the `black_scholes_price` function. This function evaluates the standard Black–Scholes formula for European options and its main goal is to be used as comparison for the Monte Carlo results assessing the Monte Carlo correctness.

4.3 Graphical Application Structure

The graphical application is implemented using MATLAB App Designer and is organized into multiple tabs, each one focusing on a specific aspect of the project. The goal of the interface is to expose the implemented models in a simple and intuitive way.

The first tab, the *Pricer* tab, is dedicated to European option pricing. In this tab the user is able to provide the relevant model parameters such as the initial asset price S_0 , strike price K , volatility, maturity, and interest rate, as well as the numerical parameters N_{paths} and N_{steps} used for the Monte Carlo simulation. The tab outputs as labels both the analytical Black–Scholes price and the Monte Carlo estimate together with the associated standard error. The tab includes graphs that show sample GBM paths and the distribution of discounted payoffs for the given parameters.

The second tab, the *Simulator* tab, focuses exclusively on the simulation of asset price paths under the GBM model. This part of the application is designed to get a direct view of the stochastic dynamics of the underlying asset. Given a set of model parameters, the tab generates multiple price trajectories and visualizes them with a histogram of the terminal asset prices, also this tab does not have any pricing logic as the paths are the main point of the tab.

As a tab a *Help/About* tab is included to provide a brief description of the purpose of the application, who made it and the authors name and to summarize the functionality of each module.

4.4 User Interaction Flow

Across all tabs the application follows a consistent interaction pattern. The user provides model and numerical parameters through the interface text edit fields which are first validated. The functions made to handle this are called and the results are displayed through numerical outputs and plots.

In the *Pricer* tab, this process leads to the evaluation of the Black–Scholes price, the Monte Carlo estimate, and the associated standard error, together with visualizations of sample paths and payoff distributions. In the *Simulator* tab, the same structure is used to generate and visualize GBM paths and terminal price distributions, remembering that its emphasis is not on pricing.

There are also additional features like the reset button for the parameters and plots and the seed fixer that enables the user to use a fix seed for reproducibility.

5 Results and Demonstration

This section presents the main results obtained using the implemented pricing and simulation methods. The objective is to verify that the numerical models behave as expected and to illustrate their output through both standalone experiments and the graphical application.

5.1 Pricing Validation

The pricing routines were first validated by comparing the Monte Carlo estimates with the analytical Black–Scholes solution for European options. For a fixed set of parameters, the Monte Carlo price approaches the Black–Scholes price as the number of simulated paths increases. This behavior is consistent with the theoretical convergence properties of Monte Carlo methods and confirms that the pricing implementation is correct.

In addition to the price estimate, the standard error reported by the Monte Carlo method decreases as the number of simulations grows, providing a quantitative indication of the estimator’s accuracy.

5.2 GBM Path Simulation

Figure 1 shows a set of asset price paths generated under the Geometric Brownian Motion model. All paths start from the same initial price and gradually diverge over time as randomness accumulates. This behavior reflects the stochastic nature of the model and illustrates how volatility influences the dispersion of possible future prices.

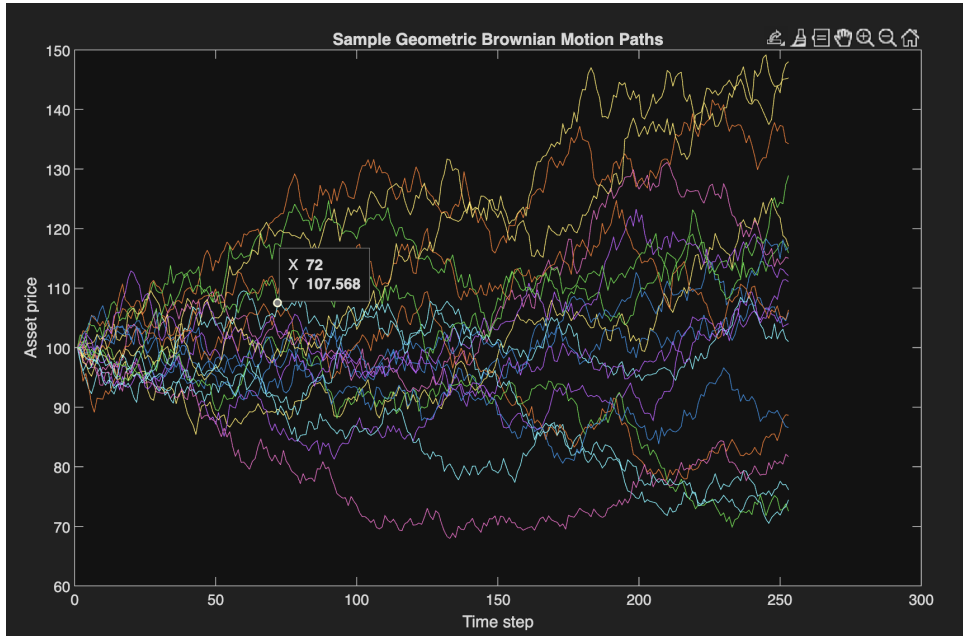


Figure 1: Sample Geometric Brownian Motion price paths generated by the simulation model.

5.3 Terminal Price Distribution

The distribution of terminal prices obtained from the Monte Carlo simulation is shown in Figure 2. The right skewed shape of the histogram is consistent with the lognormal distribution implied by the GBM model. This provides an additional sanity check on the correctness of the simulation and supports the validity of the numerical results.

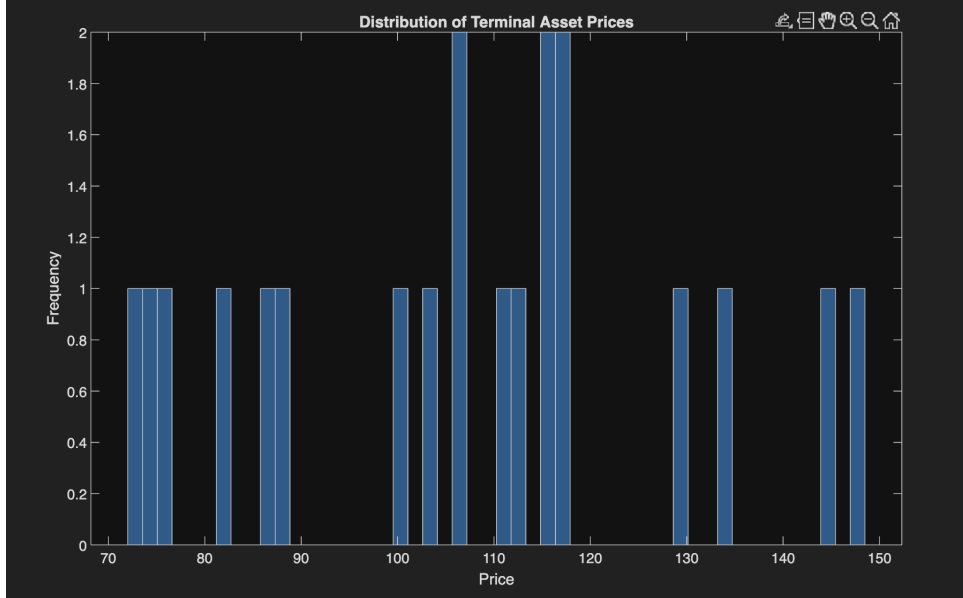


Figure 2: Histogram of terminal asset prices obtained from Monte Carlo simulation.

5.4 Graphical Application Demonstration

The validated pricing and simulation routines are integrated into an interactive graphical application developed using MATLAB App Designer. The application allows users to modify parameters through the editable numerical fields and be able to compute.

The Pricer tab displays both the analytical Black–Scholes price and the Monte Carlo estimate, together with the associated standard error and supporting visualizations. The Simulator tab focuses on the generation and exploration of GBM paths and terminal price distributions. This GUI makes it easier to explore the behavior of the models and option possible behaviors and to understand the impact of different parameters in practice, while also allowing for reproducibility.

A video demonstration of the GUI: [Demonstration Video](#).

6 Programming Challenges and Solutions

During the development of the project, a lot of problems came up, within these problems there was: numerical stability issues, performance, and the implementation of GUI functionalities with until then unknown objects of the App designer into a graphical application.

6.1 Managing Monte Carlo Performance

One of the main challenges was handling the computational cost of Monte Carlo simulations, especially when using a large number of paths and time steps. Simulating and plotting all paths simultaneously quickly lead to performance issues and even visualization problems. With a great enough number the computer was not able to obtain results in decent time for the scope of the project.

To address this, the implementation limits the number of paths displayed in the graphical interface to 30 (in the current implementation), while still allowing a large number of paths to be used internally for pricing. This takes care of numerical accuracy while allowing the app to be functional. Monte Carlo implementation was probably the main thing that I wanted to take care of correctly therefore a lot of care went into the correct implementation.

6.2 Reproducibility and Randomness

Another challenge was balancing the stochastic nature of Monte Carlo simulations with the need for reproducibility. By default, each simulation should generate different results, but it is also useful to be able to reproduce specific outcomes for validation and debugging. It was handled by introducing an optional fixed random seed that can be enabled by the user.

6.3 Input Validation and Edge Cases

Careful input validation was required to prevent invalid parameter combinations from causing numerical errors or unexpected behavior. For example, negative values for volatility or maturity, as well as invalid numerical settings for the Monte Carlo simulation, needed to be handled explicitly as they might break the function because of by 0 divisions.

Basic validation checks were added at the interface level with limits in the fields and also a function that runs before the actual mathematical functions to ensure that only admissible parameter values are passed to the methods.

6.4 Implementation of Mathematical Models

The mathematical models are standard in the options industry but implementing them correctly required careful attention as I had never done it. For the GBM simulation, the main challenge was translating the continuous time formulation into a stable discrete time update, leading me to better understand the need of discretization, ensuring that the simulated paths followed the expected behavior.

In the Monte Carlo pricing routine, care was needed when computing discounted payoffs and estimating the standard error, as small implementation mistakes can easily distort the final price estimate, at first I had a problem with transposing a matrix that was causing quite some trouble. Also in the implementation of Black-Scholes required correctly managing different numerical edge cases, such as very small maturities or volatility values, to maintain consistency with the Monte Carlo results.

7 Conclusion

This project is the development of an interactive MATLAB application for European option pricing and asset price simulation. The implementation combines the analytical and widely used Black–Scholes model with a Monte Carlo pricing approach based on Geometric Brownian Motion.

The results show that the Monte Carlo estimates converge to the Black–Scholes price as the number of simulated paths increases, and that the simulated price paths behave in line with the assumptions of the GBM model, this is what was supposed to happen.

Focusing on MATLAB and software, it was important to separate the numerical models from the graphical interface, MATLAB's built in function file handling did a great job on this, which made the implementation easier to test and modify, while also making the application more intuitive to use. Also, it proved MATLAB as a great tool for mathematical and numerical analysis and forecasting, with many built in functionalities that allow for a swift and stable development.

Overall, quite some challenges emerged from the theoretical background, but as the course has been thorough, the implementation was way simpler than it would have been without the previous knowledge obtained. The project provides a solid base that could be extended in the future to include more advanced pricing methods or option models, as well as a showcase of MATLAB's capabilities.

References

- [1] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, vol. 81, no. 3, pp. 637–654, 1973.
- [2] J. C. Hull, *Options, Futures, and Other Derivatives*, 9th ed., Pearson Education, 2015.
- [3] S. Boyd, *Monte Carlo Methods*, Stanford University Lecture Notes, https://web.stanford.edu/class/ee364a/lectures/monte_carlo.pdf, accessed 2025.
- [4] MathWorks, *MATLAB Documentation*, <https://www.mathworks.com/help/matlab/>, accessed 2025.